

TikTok 前端开发, 有点顶不住啊

TikTok 前端开发, 有点顶不住啊

一面

聊项目：

- 1、项目中所用的通信方式（http, udp, rpc）
- 2、如何大批量高效渲染列表
- 3、对于消息通知如何实时更新（出错了怎么办）

聊产品：

- 1、海外用户和国内用户有什么区别？
- 2、你认为如果你做出海业务的话，你需要从哪些方面考虑

聊基础：

- 1、事件循环
- 2、Promise
- 3、vue 如何实现响应式的（2 和 3 都说）
- 4、你会如何设计 vuex
- 5、vue 中的钩子函数

撸代码：

- 1、实现深拷贝
- 2、实现一个简单的模板替换
1. #5Promise

第二次面试

- 1、你接触过的网络协议？

- 2、http1 和 2 的区别，https 是什么？
- 3、对称加密和非对称加密算法具体是怎么样的，数字签名是什么，具体的业务场景？
- 4、常见的网络模型（每一层都是做什么的）
- 5、说一下常见的数据结构（数组，set，map），对应的业务场景
- 6、你提到 map，说一说 weak Map 和 map 的区别。
- 7、数组有哪些变体（队列，堆，栈），堆是怎么实现的。
- 8、排序算法有哪些（哪些是稳定的，哪些是不稳定的）
- 9、很多时候我们需要调度任务，你是怎么实现任务调度的
- 10、有了解过数据库吗？关系型数据库和非关系型的数据库有哪些？
- 11、关系型数据库和非关系型的数据库有什么区别？
- 12、你项目里写到你实现过 vue 的底层源码？讲一讲你的实现思路
- 13、vue 底层响应式实现逻辑，为什么要用 proxy？
- 14、vue2 用的 Object.defineProperty 不好吗？vue2 数组这一块，响应式怎么处理的？
- 15、diff 算法如何实现？
- 16、你实现的源码中是如何实现模版替换原理？
- 17、既然你谈到 attribute 和 Property 要做特殊处理，为什么？

如何做手撕代码（很简单）：

- 1、实现两边自适应，中间固定
- 2、数组扁平化且不重复（使用三种方法）
- 3、二叉树的层序遍历

写的很快，那我再回到你的项目中来吧

- 1、你写的这个插件系统，rpc 通信具体如何实现了什么，细节讲一？
- 2、为啥使用 iframe，为啥不用微前端？
- 3、你用 rpc+iframe 有考虑内存泄漏吗？
- 4、iframe 加载延迟，这个有想过如何优化吗？
- 5、你在这个项目中的成就？你遇到过哪些困难，如何解决的？

参考答案

一面

聊项目

1. 项目中所用的通信方式：

在项目中，我们可能会根据具体需求采用不同的通信方式。例如，对于 Web 应用，HTTP 和 HTTPS 是常用的通信协议，它们支持客户端和服务端之间的数据传输。

UDP（用户数据报协议）则是一种无连接的通信协议，适用于需要快速传输且对可靠性要求不高的场景。

RPC（远程过程调用）则是一种允许程序在网络中远程执行代码的技术，它隐藏了底层的通信细节，使得开发者可以像调用本地方法一样调用远程服务。

2. 如何大批量高效渲染列表：

对于大批量数据的渲染，我们可以采用分页或虚拟滚动等技术来优化性能。分页技术将大量数据分成多个页面，每次只渲染当前页面的数据。

虚拟滚动则通过只渲染可见区域的数据，并在用户滚动时动态更新渲染的数据集，从而大大减少了 DOM 节点的数量，提高了渲染效率。

3. 对于消息通知如何实时更新（出错了怎么办）：

对于消息通知的实时更新，我们可以使用 WebSocket 等技术建立长连接，服务器可以主动向客户端推送消息。

如果出现错误，我们可以采用重试机制、降级处理或向用户展示错误提示等方式进行处理。同时，我们也需要建立完善的日志和监控体系，以便及时发现和解决问题。

聊产品

1. 海外用户和国内用户有什么区别：

海外用户和国内用户在文化、语言、使用习惯等方面存在差异。例如，海外用户可能更注重产品的国际化程度、隐私保护和数据安全等方面。

在产品的设计时，我们需要充分考虑这些差异，提供符合海外用户需求的功能和服务。

2. 你认为如果你做出海业务的话，你需要从哪些方面考虑：

做出海业务时，我们需要考虑目标市场的文化、语言、法律法规等因素，确保产品符合当地的规定和习惯。

同时，我们还需要考虑产品的本地化、国际化程度以及目标市场的竞争情况等因素，制定相应的市场策略和推广计划。

聊基础

1. 事件循环：

事件循环是 JavaScript 或 Node.js 为解决单线程代码执行不阻塞主进程而引入的一种机制。它负责执行代码、收集和处理事件以及执行队列中的子任务。

在事件循环中，所有任务被分为同步任务和异步任务。同步任务在主线程中按顺序执行，而异步任务则被放入任务队列中等待执行。当主线程空闲时，事件循环会从任务队列中取出任务并执行。

2.Promise：

Promise 是 JavaScript 中用于处理异步操作的对象。它代表了一个可能现在、也可能将来才会知道结果的操作。

Promise 有三种状态：pending（进行中）、fulfilled（已成功）和 rejected（已失败）。通过 Promise 对象，我们可以更加优雅地处理异步操作，避免回调地狱等问题。

3.vue 如何实现响应式的：

Vue.js 通过数据劫持（如使用 `Object.defineProperty`）和发布者 - 订阅者模式实现了响应式系统。

当 Vue 实例的数据发生变化时，它会触发相应的 setter 函数，通知依赖该数据的组件重新渲染。

同时，Vue 还提供了计算属性和侦听器等功能，以便开发者更加方便地处理复杂的数据逻辑和响应式更新。

Vue 的响应式原理：

Vue 使用 ES6 的 Proxy 特性（在 Vue2.x 中主要通过 `Object.defineProperty` 来实现）来劫持 data 中所有属性的 getter 和 setter。

当这些属性在模板中被用作渲染数据时，Vue 会将它们标记为依赖。

当这些属性的值发生变化时，Vue 会触发相应的更新机制，通知依赖它们的组件重新渲染。

4. 你会如何设计 vuex：

在设计 Vuex 时，我会遵循模块化、单一职责和不可变数据等原则。

我会将状态划分为多个模块，每个模块负责自己的状态和业务逻辑。同时，我会确保每个模块的功能单一且清晰，以便于维护和扩展。

在处理状态时，我会采用不可变数据的方式，以避免直接修改状态带来的副作用和调试困难等问题。

5.vue 中的钩子函数：

Vue 中的钩子函数提供了丰富的生命周期事件，允许开发者在不同的阶段执行自定义的逻辑。

常用的钩子函数包括 `beforeCreate`、`created`、`beforeMount`、`mounted`、`beforeUpdate`、`updated`、`beforeDestroy` 和 `destroyed` 等。

这些钩子函数可以帮助开发者更好地理解和控制组件的生命周期，从而实现更加高效和可靠的应用开发。

撸代码

1. 实现深拷贝：

java

复制代码

```
1 // Java实现深拷贝的示例代码
2 public class User implements Cloneable, Serializable {
3     private Integer id;
4     private Integer age;
5     private String name;
6     private Car car;
7     private String category;
8
9     // Getters and Setters省略
10
11    @Override
12    protected Object clone() throws CloneNotSupportedException {
13        // 使用序列化实现深拷贝
14        try (ByteArrayOutputStream bos = new ByteArrayOutputStream();
15             ObjectOutputStream oos = new ObjectOutputStream(bos);
16             ByteArrayInputStream bis = new ByteArrayInputStream(bos.toByteArray());
17             ObjectInputStream ois = new ObjectInputStream(bis)) {
18            oos.writeObject(this);
19            return ois.readObject();
20        } catch (IOException | ClassNotFoundException e) {
21            throw new CloneNotSupportedException(e);
22        }
23    }
24 }
25
26 class Car implements Serializable {
27     private Integer id;
28     private String color;
29     private String name;
30
31     // Getters and Setters省略
32 }
```

2. 实现一个简单的模板替换：

java

复制代码

```
1 // Java实现简单模板替换的示例代码
2 public class TemplateReplacer {
3     public static String replace(String template, Map<String, String> data) {
4         for (Map.Entry<String, String> entry : data.entrySet()) {
5             template = template.replace("${" + entry.getKey() + "}", entry.getValue());
6         }
7         return template;
8     }
9
10    public static void main(String[] args) {
11        String template = "Hello, ${name}! You have ${count} new items";
12        Map<String, String> data = new HashMap<>();
13        data.put("name", "Alice");
14        data.put("count", "5");
15
16        String result = replace(template, data);
17        System.out.println(result); // 输出: Hello, Alice! You have 5 new items
18    }
19 }
```

3. #5 Promise (JavaScript 实现) :

```
1 // JavaScript实现Promise的示例代码
2 new Promise((resolve, reject) => {
3     // 模拟异步操作
4     setTimeout(() => {
5         const success = true; // 可以根据需要修改为false以测试reject
6         if (success) {
7             resolve("操作成功! ");
8         } else {
9             reject("操作失败! ");
10        }
11    }, 1000);
12 })
13 .then(result => {
14     console.log(result); // 如果成功，则输出“操作成功!”
15 })
16 .catch(error => {
17     console.error(error); // 如果失败，则输出“操作失败!”
18 });
```

二面

2.HTTP1 和 HTTP2 的区别，HTTPS 是什么？

- HTTP/1.1：逐个请求响应，每个请求都会有独立的 TCP 连接，容易造成网络拥堵。支持管道化但不广泛应用。
- HTTP/2：基于单个 TCP 连接的多路复用，允许并行发送多个请求。引入二进制分帧层，头部压缩（HPACK）和服务器推送等特性，极大提高了性能。
- HTTPS：在 HTTP 基础上加入 SSL/TLS 协议，实现数据加密传输，确保数据的保密性、完整性和身份验证。

3. 对称加密和非对称加密算法，数字签名，业务场景

- 对称加密：使用同一个密钥进行加密和解密，如 AES、DES。适用于大数据量的高速加密场景，如数据存储加密。

- 非对称加密：使用公钥加密，私钥解密，如 RSA、ECC。适用于密钥分发和身份验证，如 SSL/TLS 握手。
- 数字签名：通过私钥对数据生成签名，验证方用公钥验证签名，确保数据未被篡改，常用于软件分发、电子合同等场景。

4. 常见的网络模型

- OSI 七层模型：
 - 物理层：传输原始比特流（如光纤、电缆）
 - 数据链路层：节点间数据帧传输（如以太网）
 - 网络层：路由选择和数据包传输（如 IP）
 - 传输层：端到端通信和错误修复（如 TCP/UDP）
 - 会话层：管理会话和连接（如 RPC）
 - 表示层：数据格式转换（如加密解密）
 - 应用层：网络服务与终端用户交互（如 HTTP/FTP）

5. 常见的数据结构

- 数组：有序、索引访问。适用于需要频繁随机访问的场景。
- Set：无序、唯一元素。适用于需要去重的场景，如处理集合操作。
- Map：键值对存储。适用于需要快速查找的场景，如缓存。

6. WeakMap 和 Map 的区别

- Map：强引用，键和值都被引用，除非手动删除，否则不会被垃圾回收。
- WeakMap：弱引用，键只能是对象，键的引用不计入垃圾回收引用链，适用于缓存等场景。

7. 数组的变体（队列，堆，栈），堆的实现

- 队列：先进先出（FIFO），如任务调度。
- 堆：完全二叉树，实现优先队列。
- 栈：后进先出（LIFO），如函数调用。

- 堆实现：通常用数组表示完全二叉树，通过索引计算父子节点关系。

8. 排序算法

- 稳定排序：冒泡排序、插入排序、归并排序、基数排序。
- 不稳定排序：快速排序、选择排序、堆排序。

9. 任务调度

- 定时器：使用 `setTimeout` 或 `setInterval`。
- 任务队列：使用消息队列系统如 RabbitMQ，Kafka。

10. 关系型数据库和非关系型数据库

- 关系型数据库：如 MySQL、PostgreSQL，基于表的结构，支持 SQL。
- 非关系型数据库：如 MongoDB、Redis，支持灵活的数据模型。

11. 关系型和非关系型数据库的区别

- 关系型：结构化数据，支持事务，数据一致性高。
- 非关系型：灵活的数据结构，适用于大规模数据和高并发场景，通常不支持事务。

13.Vue 底层响应式实现逻辑，为什么要用 Proxy?

- Vue2：使用 `Object.defineProperty`，无法监听数组变动和新增属性。
- Vue3：使用 `Proxy`，能够直接拦截对象操作，性能更好，支持全面的变动监听。

14.Vue2 用的 `Object.defineProperty` 不好，数组响应式怎么处理?

- Vue2 数组处理：通过重写数组的变异方法（如 `push`、`splice`）实现响应式。

15.Diff 算法实现

- 算法步骤：
 - 比较新旧虚拟 DOM 树。
 - 找出节点的最小变化集合，生成补丁。
 - 应用补丁更新真实 DOM。

16. 模板替换原理

- 实现思路：使用正则表达式匹配模板中的占位符，将其替换为实际数据。

17.Attribute 和 Property 的区别

- Attribute：HTML 属性，反映初始值。
- Property：DOM 属性，反映当前状态。
- 处理方式：手动区分需要更新的内容，用适当的方式进行设置。